INGENIERÍA DE SOFTWARE · UADE

Diseño del frontend

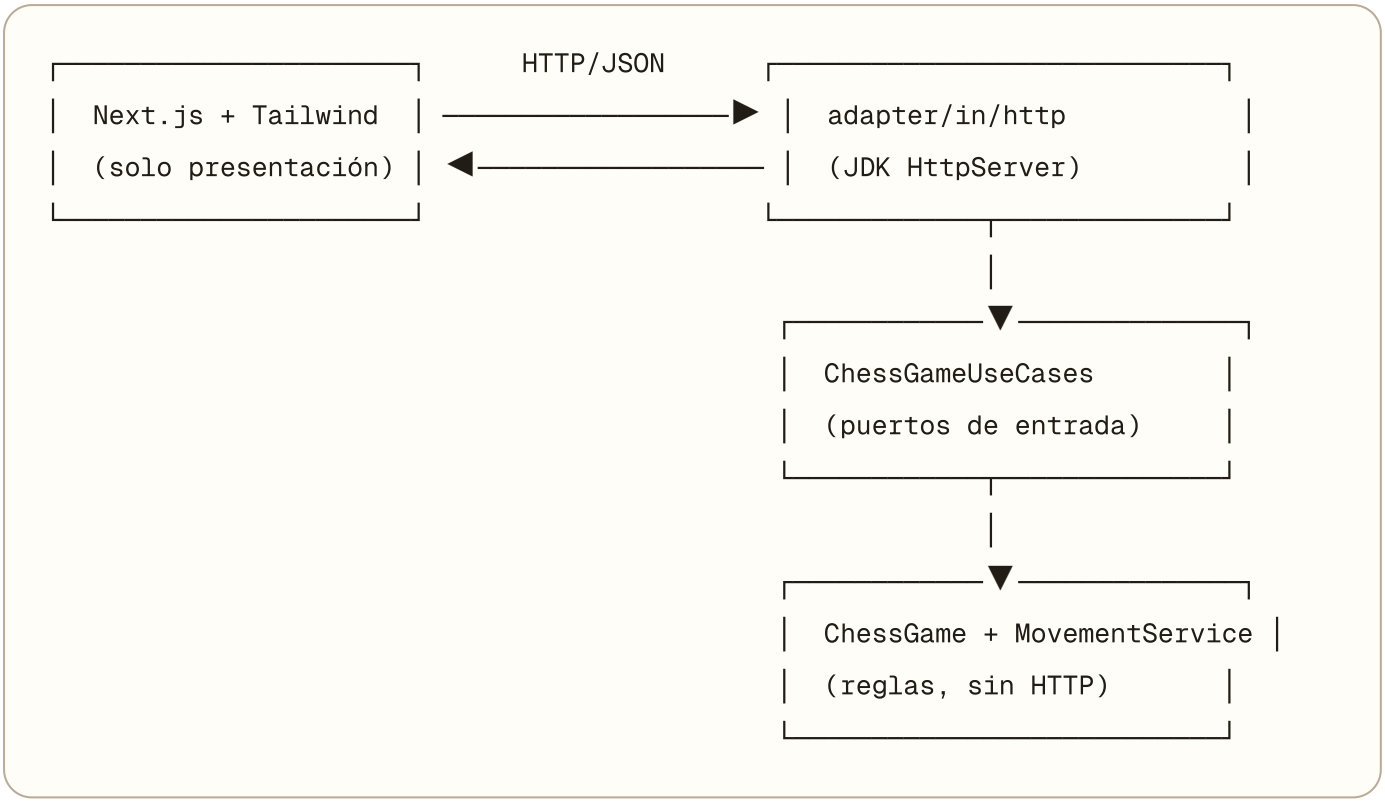
Contrato de la API, pantallas, temas y criterios visuales de la web.

Medina Cotos · Tortora · Umerez · Quintana

Este documento define **cómo** conectamos el frontend Next.js con el núcleo Java sin romper la arquitectura hexagonal. El frontend consume casos de uso a través de un adaptador HTTP nuevo; no reimplementa ninguna regla del ajedrez.

> **Estado:** implementado. Adaptador HTTP, frontend Next.js y pulido visual están hechos y verificados (pasos 1 a 7 del orden de implementación).

1. Idea central



Reglas de oro:

- **El front no valida jugadas.** Solo pinta `GameView` y manda `from` / `to`.
- **El dominio no cambia.** La única pieza nueva en Java es un adaptador de entrada.
- **Un solo contrato.** El JSON es exactamente el `GameView` que ya existe.
- **Consola y web conviven.** Es la prueba de que la arquitectura funciona.

---

2. Adaptador HTTP en Java (sin frameworks)

Usamos `com.sun.net.httpserver.HttpServer`, que viene en el JDK: cero dependencias, coherente con el resto del TP.

```
src/main/java/com/uade/ajedrez/infrastructure/adapter/in/http/
├── HttpGameServer.java    # arranque, rutas, CORS, manejo de errores
├── GameViewJson.java      # GameView -> JSON (escritor manual, ~80 líneas)
├── JsonBody.java          # lectura mínima de { "from": "e2", "to": "e4" }
├── Json.java              # escape de strings y helpers de JSON
└── HttpError.java         # mapea excepciones de aplicación a códigos HTTP
```

`App` elige el adaptador:

POWERSHELL

```
java -cp bin com.uade.ajedrez.App            # consola (actual)
java -cp bin com.uade.ajedrez.App --web      # HTTP en http://localhost:8080
```

Endpoints

Método	Ruta	Body	Respuesta
`GET`	`/api/games`	—	`200` + `{ "games": [GameSummary] }`
`POST`	`/api/games`	`{ "whitePlayerName": "...", "blackPlayerName": "..." }`	`201` + `GameView`
`GET`	`/api/games/{id}`	—	`200` + `GameView`
`POST`	`/api/games/{id}/moves`	`{ "from": "e2", "to": "e4" }`	`200` + `GameView`
`GET`	`/api/games/{id}/legal-moves?from=e2`	—	`200` + `{ "moves": ["e3","e4"] }`
`POST`	`/api/games/{id}/abandon`	—	`200` + `GameView`
`POST`	`/api/games/{id}/save`	—	`200` + `GameView`

Método	Ruta	Body	Respuesta
`GET`	`/api/games/{id}/replay?position=N`	—	`200` + `ReplayView` (`400` si la partida no terminó)
`GET`	`/api/health`	—	`200` + `{ "status": "ok" }`

Formato de `GameView`

JSON

```
{
  "id": "g_8f2c...",
  "status": "IN_PROGRESS",
  "turn": "WHITE",
  "turnNumber": 1,
  "whitePlayer": { "name": "Blanco", "color": "WHITE" },
  "blackPlayer": { "name": "Negro", "color": "BLACK" },
  "board": [
    [ {"type":"ROOK","color":"BLACK"}, {"type":"KNIGHT","color":"BLACK"}, null, "...
    "..."
  ],
  "history": [ { "from": "e2", "to": "e4" } ],
  "lastMove": { "from": "e2", "to": "e4" },
  "winner": null
}
```

- `board` es una matriz de 8×8, `board[0][0]` = **a8**, `board[7][7]` = **h1**. Las casillas vacías son `null`.

- `winner` es `null` salvo en `CHECKMATE`. En `ABANDONED` y `DRAW` también es `null`; el resultado se muestra a partir de `status`.

Formato de `ReplayView`

El replay devuelve el tablero de una posición del historial junto con el resultado final de la partida:

JSON

```
{
  "game": { "id": "g_8f2c...", "status": "IN_PROGRESS", "turn": "BLACK", "board": ["
  "position": 3,
  "totalMoves": 8,
  "result": "CHECKMATE"
}
```

- `position` viene clampeada entre `0` y `totalMoves`, así que pedir `-5` o `999` devuelve un extremo válido.
- `game.status` es el estado de la posición que se está viendo; `result` es el final real de la partida.
- Solo hay replay para partidas terminadas: `CHECKMATE`, `ABANDONED` o `DRAW`. Una partida `IN_PROGRESS` o `CHECK` responde `400`.

Formato de `GameSummary`

`GET /api/games` no devuelve el tablero: alcanza para reconocer la partida en el lobby.

JSON

```
{
  "games": [
    {
      "id": "g_8f2c...",
      "status": "IN_PROGRESS",
      "turn": "BLACK",
      "turnNumber": 1,
      "whitePlayer": "Ana",
      "blackPlayer": "Beto",
      "moveCount": 1,
      "lastMove": { "from": "e2", "to": "e4" },
      "updatedAt": 1762000000000
    }
  ]
}
```

- `updatedAt` son milisegundos desde epoch; sale de la fecha del archivo.
- Las partidas vienen de la más reciente a la más vieja.
- Los archivos corruptos se ignoran para no romper el listado completo.

Errores

JSON

```
{ "error": "MOVE_REJECTED", "message": "Cant move enemy piece" }
```

Situación	Código
Body inválido / id inválido	`400`
Partida inexistente	`404`
Movimiento ilegal o partida terminada	`409`
Falla interna	`500`

CORS abierto (`*`) para desarrollo; se restringe al dominio del frontend cuando lo definamos.

---

3. Frontend Next.js

Carpeta `web/` en la raíz del repo (el backend Java queda intacto).

```
web/
├── package.json
├── next.config.ts              # loader de Tailwind para Turbopack
├── tsconfig.json
├── .env.local.example          # NEXT_PUBLIC_API_URL=http://localhost:8080
└── src/
    ├── app/
    │   ├── layout.tsx          # shell: header, fondo, tipografía
    │   ├── globals.css         # Tailwind + variables de tema
    │   ├── page.tsx            # Lobby: crear y abrir partidas guardadas
    │   ├── docs/page.tsx       # índice de documentación
    │   ├── docs/[slug]/page.tsx # documento markdown
    │   ├── game/[id]/page.tsx  # Pantalla de juego
    │   └── game/[id]/replay/page.tsx  # Pantalla de replay
    ├── components/
    │   ├── board/
    │   │   ├── ChessBoard.tsx  # grilla 8x8 + interacción
    │   │   ├── Square.tsx      # casilla, coordenadas, resaltados
    │   │   └── Piece.tsx       # glifo de pieza
    │   ├── docs/
    │   │   ├── DocsScreen.tsx  # layout de la documentación
    │   │   ├── DocsNav.tsx     # menú de documentos
    │   │   ├── DocsToc.tsx     # índice del documento
    │   │   └── DocMarkdown.tsx # render de markdown y links internos
    │   ├── panels/
    │   │   ├── PlayerCard.tsx  # nombre, color, capturadas, turno
    │   │   ├── MoveHistory.tsx # lista de jugadas en pares
    │   │   ├── GameStatus.tsx  # estado, turno, acciones
    │   │   └── ResultBanner.tsx
    │   ├── lobby/
    │   │   └── LobbyScreen.tsx # crear y abrir partidas guardadas
    │   ├── game/
    │   │   ├── GameScreen.tsx  # layout de la partida y atajos
    │   │   └── ReplayScreen.tsx # tablero, historial y controles del replay
    │   └── ui/
    │       ├── Button.tsx
    │       ├── Input.tsx
```

```
│       └── Toast.tsx
├── hooks/
│   └── useGame.ts          # estado + polling + acciones
└── lib/
    ├── api.ts              # cliente tipado de la API
    ├── types.ts            # GameView, GameSummary, Color, PieceType, GameState
    ├── status.ts           # etiquetas y colores del estado
    ├── docs.ts             # lista, lectura y links de los documentos
    ├── markdown.ts         # slugs y extracción de títulos
    └── board.ts            # helpers de coordenadas y agrupado de historial
```

Responsabilidades

Archivo	Hace	No hace
`lib/api.ts`	`fetch` + parseo + errores tipados	reglas del juego
`hooks/useGame.ts`	estado, polling, mutaciones	dibujar
`ChessBoard.tsx`	layout y clicks	saber si el movimiento es válido
`ReplayScreen.tsx`	pedir posiciones y navegar el historial	reconstruir jugadas o validar reglas
`lib/docs.ts`	listar, leer y enlazar los documentos	renderizar markdown
`DocMarkdown.tsx`	renderizar markdown y sus links internos	decidir qué documentos existen
`Piece.tsx`	glifo/color	lógica

Flujo de una jugada

1. Click en pieza propia → `GET /legal-moves?from=e2` → se marcan puntos.
2. Click en destino marcado → `POST /moves`.
3. El servidor devuelve el `GameView` nuevo → se reemplaza el estado.
4. Si el servidor rechaza (`409`) → toast con el mensaje y se mantiene la selección.

No hay ajedrez en TypeScript: **el front nunca decide si un movimiento es legal**.

Sincronización

`useGame` hace `GET /api/games/{id}` cada 2 s mientras la pestaña está visible (`document.visibilityState`). Sirve para "hotseat" en dos dispositivos y para retomar una partida abierta en otra pestaña. Si más adelante hace falta, se cambia por SSE sin tocar los componentes.

Replay

El replay está disponible para partidas terminadas por **jaque mate**, **abandono** o **tablas**. Se llega desde el overlay de fin de partida (**Ver replay**) o desde el enlace **Replay** que aparece en el lobby para esas partidas.

`ReplayScreen` pide al backend una posición por vez con `GET /api/games/{id}/replay?position=N`. No reconstruye jugadas ni guarda historial local: muestra el `GameView` que devuelve el servidor y mueve el puntero. Los controles son **Inicio**, **Anterior**, **Siguiente** y **Final**; también responden las flechas ← →, `Inicio` y `Fin`. El panel derecho muestra el estado de la posición actual y, separado, el resultado final de la partida.

Documentación

`/docs` muestra los documentos del repositorio: `README.md`, `web/README.md` y los archivos de `docs/`. `lib/docs.ts` tiene la lista fija de documentos, lee cada archivo con `node:fs` y traduce los enlaces entre `.md` a rutas de la web; `DocMarkdown.tsx` los

renderiza con `react-markdown` y `remark-gfm`.

La pantalla tiene el menú de documentos, el contenido y un índice lateral con los `h2` y `h3`. Los ids de los títulos salen de `lib/markdown.ts`, así los enlaces del índice funcionan sin JavaScript. La lista es fija para que un slug inventado no pueda leer otra parte del disco. Como los documentos están fuera de `web/`, `lib/docs.ts` resuelve la raíz del repo desde `process.cwd()`.

---

4. Diseño visual

Estética **"club de ajedrez nocturno"**: fondo profundo, madera cálida en el tablero, acento dorado. Sobrio, sin animaciones exageradas.

Paleta

Rol	Hex	Uso
`bg`	`#0B0F14`	fondo de la app
`surface`	`#121820`	tarjetas y paneles
`border`	`#1F2A37`	separadores
`text`	`#E6EDF3`	texto principal
`muted`	`#8B98A9`	texto secundario
`accent`	`#D9A441`	turno activo, botones primarios
`danger`	`#E5484D`	jaque, abandono
`success`	`#3FB950`	victoria
`light-square`	`#EDD6B0`	casillas claras
`dark-square`	`#B58863`	casillas oscuras
`highlight`	`#D9A44155`	último movimiento / selección

Las casillas se definen como variables CSS para poder ofrecer un tema "tournament green" (`#EEEED2` / `#769656`) más adelante.

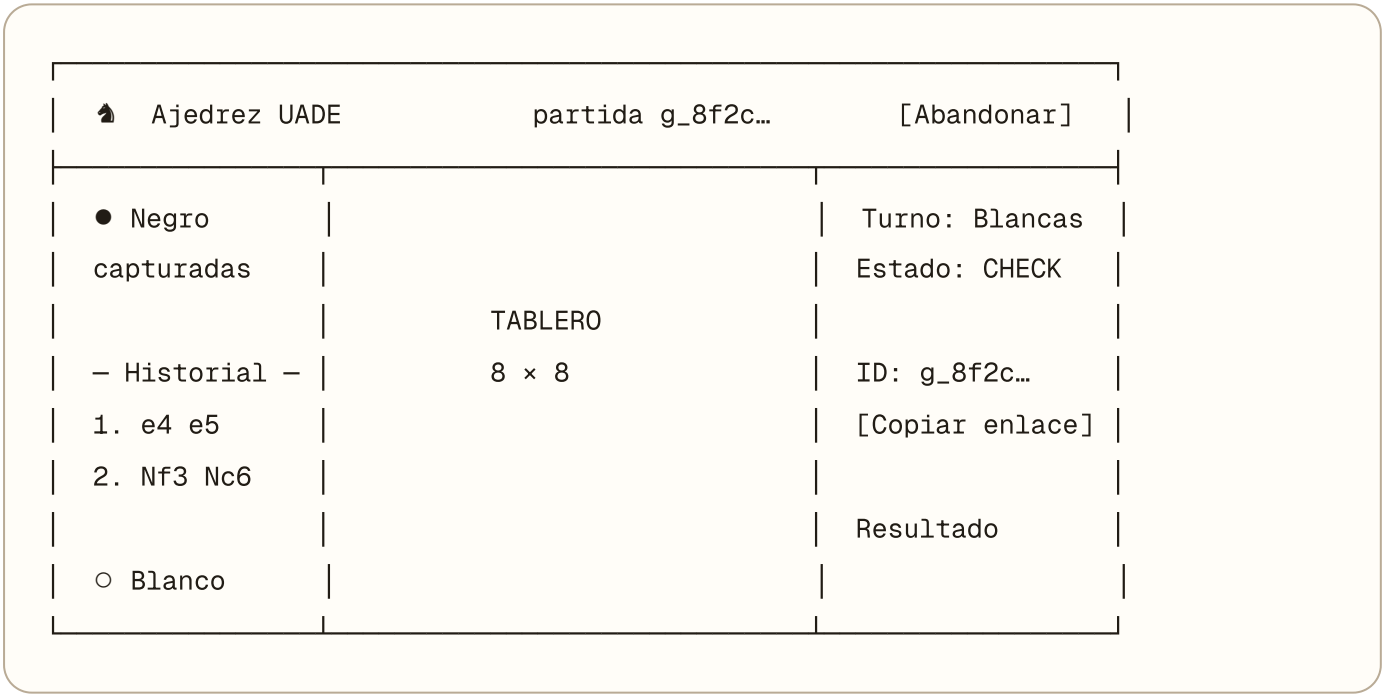
Tipografía

- UI: **Inter** (o `Geist` de Next) — pesos 400/500/600.
- Piezas: glifos Unicode ♔♕♖♗♘♙ / ♚♛♜♝♞♟ con `Segoe UI Symbol` como fallback. Cero assets. Si más adelante se quieren piezas más finas, se reemplaza solo `Piece.tsx` por SVG.

Temas

- **Oscuro por defecto**. El claro reutiliza los mismos tokens `ink-*`, `gold-*`, `danger` y `success`, redefinidos sobre papel cálido; los componentes no cambian.
- El tema vive en `<html data-theme="light|dark">`. El botón ☾/☀ del header cambia el atributo y guarda la preferencia en una cookie.
- El layout lee la cookie en el servidor y renderiza el atributo correcto desde el primer HTML, así que no hay flash ni script de inicialización.
- El tablero conserva la madera en ambos temas. Cambian superficies, textos, bordes, piezas capturadas y el velo del resultado.

Layout de escritorio (≥ 1024 px)



- Tablero máximo `min(66vh, 600px)`, cuadrado, con coordenadas en el borde.
- El panel izquierdo muestra negras arriba y blancas abajo (según la orientación del tablero).
- El panel derecho concentra estado, ID de partida (con botón para copiar el enlace) y acciones.

Layout móvil (< 1024 px)

Se apila en este orden: cabecera → tablero a ancho completo → tarjetas de jugadores con el historial en el medio → panel de estado y acciones. El tablero va primero para que la partida quede visible sin scrollear.

Estados visuales del tablero

Estado	Cómo se ve
Casilla seleccionada	borde/velo dorado
Destino legal	punto de 28 % del tamaño de la casilla
Destino con captura	anillo alrededor de la pieza
Último movimiento	fondo `highlight` en origen y destino
Rey en jaque	halo rojo en la casilla del rey

Lobby (`/`)

- Título y subtítulo.
- Tarjeta **Nueva partida**: dos inputs (blancas / negras), botón primario.
- Sección **Partidas guardadas**: lista las partidas de `games/` con jugadores, estado, turno, cantidad de jugadas, última jugada y fecha. Un clic abre la partida. Las partidas terminadas muestran además un enlace **Replay**. Tiene botón para actualizar el listado.
- Si el backend no responde: aviso claro con el comando para levantarlo.

---

5. Orden de implementación

1. **Backend**: `GameViewJson`, `HttpGameServer`, ruteo, CORS, `App --web`.
2. **Contrato**: prueba manual con `curl` / `Invoke-RestMethod` de los 5 endpoints.
3. **Frontend base**: `create-next-app` en `web/`, tema y `lib/api.ts`.
4. **Lobby** (`/`).
5. **Tablero** (`/game/[id]`): render, selección, jugadas, historial.
6. **Pulido**: polling, toasts, estados de fin de partida, responsive.
7. **README**: cómo levantar backend y frontend juntos.

Los siete pasos están completos. Los pasos 1–2 no tocan el frontend; los pasos 3–7 no tocan Java salvo ajustes del contrato.

---

6. Fuera de alcance (por ahora)

- Autenticación y partidas multiusuario reales.
- Enroque, promoción y captura al paso (no existen en el dominio todavía).
- Websockets. El polling alcanza para la demo del TP.
- Deshacer (`MoveCommand`) desde la web: primero hay que integrarlo en `ChessGame`.
